Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 8. Seguridad en fase de producción de las aplicaciones web

Índice

Esquema

Ideas clave

8.1. Introducción y objetivos

8.2. Seguridad en la fase de producción online

8.3. Mecanismos de protección

8.4. Operaciones de seguridad

8.5. Referencias bibliográficas

A fondo

Firewall de aplicaciones web ModSecurity

Herramientas de protección en tiempo real (RASP)

WAF contra RASP

SANS Institute

Normativa y noticias al día de ciberseguridad

Test

© Universidad Internacional de La Rioja (UNIR)

Esquema



8.1. Introducción y objetivos

En la fase de producción de una aplicación web es necesario acometer las operaciones de seguridad del ciclo de vida de desarrollo seguro, que incluyen tareas de administración de seguridad, de monitorización, actualización de *software,* etc.

Una **adecuada formación y concienciación** continua en todo lo que se refiere a aspectos de seguridad es imprescindible y fomenta en la empresa una **cultura por la seguridad** acorde con los tiempos que corren, cuando cada vez parece que surgen más y **nuevas amenazas** que requieren estar prevenidos y preparados para hacerles frente en la mejor medida posible.

Cuando se materializa una amenaza hay que detectarla y, en función de su naturaleza, actuar en consecuencia de tal manera que el **daño potencial para la empresa se minimice al máximo** y el impacto sufrido por sus activos sea el menor posible.

El impacto es directamente proporcional al coste económico. Se debe diseñar y entrenar un procedimiento de respuesta y gestión de incidentes de seguridad.

En el apartado de la seguridad han de intervenir y estar concienciados **todos los elementos de la organización.** De nada sirve invertir ingentes cantidades de dinero en equipamiento como *firewalls,* sistemas de detección de intrusos (IDS), etc., si luego su configuración no se realiza de forma segura de acuerdo con estándares o si alguien suministra información aparentemente no importante a alguien que tampoco lo parecía o deja información al alcance de terceros por descuido.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Todas estas **negligencias** que pueden suponer **fugas en la seguridad** son la causa, finalmente, de grandes pérdidas económicas y de competitividad para la organización. Para paliar las pérdidas de información o su indisponibilidad, es necesario disponer y entrenar los **procedimientos de *back-up* y recuperación** que sean necesarios.

La **concienciación** en seguridad es **fundamental** en todos los estamentos de la organización. Normalmente, cualquier medida que implique concienciar al personal o implantar medidas o procedimientos de seguridad en cualquier parte de un área o departamento causa fastidio o rechazo por parte del personal que ha de observarlas. La mejor forma de tener éxito en este campo es intentar conseguir que **cada individuo entienda y comprenda la importancia** de la medida, que tiene que ver, finalmente, con alcanzar los objetivos de la organización y que el fallo o descuido personal tiene incidencia en lo colectivo.

Se debe **diseñar, mantener y revisar la seguridad de los clientes** (navegadores), de los **servidores web** y de las aplicaciones y las **conexiones entre ellos** de acuerdo con el conocimiento de seguridad adquirido, apoyándose en guías de configuración segura de estándares, y mantenerse y supervisarse periódicamente en fase de producción sin fallos.

Además de conseguir y mantener viva la concienciación de los usuarios en materia de seguridad, es importante aplicar medidas específicas relativas a la **seguridad de la documentación** de las aplicaciones web y de la **seguridad física** de las instalaciones, despachos, centro de proceso de datos, etc. Todas estas medidas deben estar incluidas en la **política de seguridad** de la organización.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Los objetivos de este tema son:

- Conocer las principales medidas de protección *online*, específicas, que se pueden implementar para proteger una aplicación web desplegada en un entorno de producción.
- Aprender las características, modos de instalación y posibilidades de protección de los *firewalls* de aplicaciones web y conocer qué tipos de pruebas hay que realizar con un *web application firewall* (WAF) para el paso de fase de pruebas a producción.
- Conocer las principales cabeceras de seguridad para implementar en una aplicación web.
- Conocer las principales vulnerabilidades y defensas de las aplicaciones que usan tecnología AJAX.

© Universidad Internacional de La Rioja (UNIR)

8.2. Seguridad en la fase de producción online

Específicamente, las actividades de seguridad que hay llevar a cabo en la **fase de producción** de la aplicación web son las denominadas **operaciones de seguridad.** Las operaciones de seguridad que hay que llevar a cabo en la aplicación web en fase de producción incluyen:

- Aplicar y supervisar medidas de protección: balanceo de carga, WAF, Real Analysis Self Protection (RASP).
- Auditoría y monitorización continua.
- Gestión de backup y recuperación, centro de respaldo.
- Gestión de incidentes de seguridad.
- Formación continua.
- Concienciación de usuarios.
- Seguridad física, del personal y de la documentación.

Ataques de denegación de servicio

Normalmente, los ataques de denegación de servicio o de denegación de servicio distribuido (*denial of service* o *distributed denial of service*, DoS-DDoS) se llevan a cabo por **inundación de peticiones** que aprovechan debilidades de los protocolos o vulnerabilidades de seguridad, como SQLi.

Los ataques de denegación de servicio de tipo **volumétrico** o de inundación de peticiones basados en el **protocolo TCP/IP** se basan en una característica o **debilidad de la conexión TCP** que permite establecer conexiones a «medio hacer» llevando a cabo las dos primeras fases y nunca la tercera y última.

© Universidad Internacional de La Rioja (UNIR)

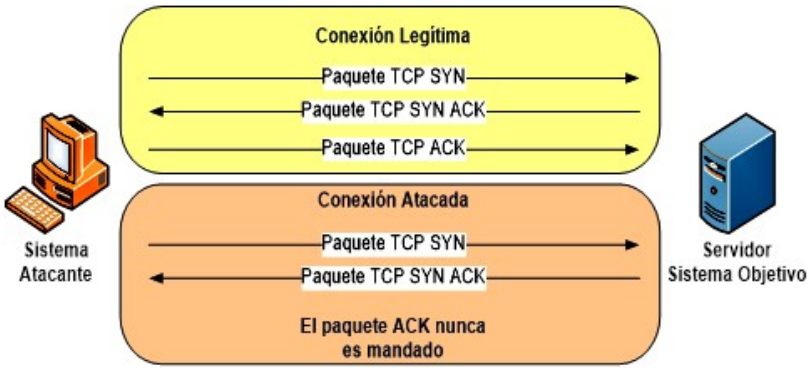


Figura 1. Conexión TCP/IP a «medio hacer». Fuente: elaboración propia.

Un ataque **DDoS** se realiza **desde varias máquinas hacia un mismo objetivo** de forma coordinada bajo la orden del atacante que controla el sistema distribuido formado por máquinas de tipo *master, slaves* y *reflectors*.

Los principales tipos de ataques DoS-DDoS específicamente a **aplicaciones web** son los siguientes:

- **Slowloris*:*** las peticiones al servidor se realizan enviando las cabeceras de la petición muy lentamente, sin terminar nunca de ser enviadas. De esta manera, mientras el servidor no recibe todas las cabeceras, no considera como sesión web establecida y no sabe si ha superado el número de conexiones máximas configuradas (por ejemplo, en Apache mediante la variable MaxClients).

- ***Slow* HTTP POST:** se envían al servidor las peticiones POST con todas las cabeceras, incluyendo *content-length*, que indica el tamaño del POST DATA a enviar. Así, el servidor queda esperando a que el cliente envíe tantos *bytes* como indique la cabecera *content-length*. El ataque en sí se basa en que el cliente irá enviando muy despacio los datos del POST hacia el servidor, consumiendo una sesión abierta por cada POST, así como los recursos que este reserva para recibir los datos que envía el cliente. Un ejemplo de herramienta de este tipo es SwitchBlade de OWASP.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Para llevar a cabo este tipo de ataques, existen herramientas como SlowhttpTest, GoldenEye o Hping3.

Las defensas específicas que se pueden implementar en el sistema operativo frente a ataques DDoS son:

- Habilitar ***SYN cache.*** Se basa en reducir la cantidad de información almacenada cuando se recibe el paquete SYN en vez de crear el bloque de control de la transmisión (*transmission control block*, TCB) completo, lo que permite soportar un mayor número de conexiones en estado *half-open*.
- Habilitar ***SYN cookies.*** No se crea el TCB hasta que ha finalizado el inicio de la conexión TCP.
- Deshabilitar la respuesta de **paquetes *ICMP destination unreachable*** (UDP).
- Reducir el tiempo *SYN received.*
- Permitir sobrescribir las conexiones *half-open.*

Las defensas específicas que se pueden implementar en la arquitectura de la aplicación web frente a ataques DDoS son:

- Usar **balanceadores de carga** configurados en modo *delayed binding* para impedir que las peticiones HTTP se entreguen a los servidores web hasta que los balanceadores hayan recibido la cabecera por completo, lo que evita que los ataques SlowHTTP lleguen a los servidores web. En el caso de SlowHTTP POST, que sí envía la cabecera correctamente, los balanceadores no mitigan este tipo de ataque.
- Usar ***clustering* de servidores** de aplicaciones web.
- Aumentar el **número máximo de conexiones** soportadas.
- Limitar el **tiempo de vida** de las conexiones a un valor razonable.

© Universidad Internacional de La Rioja (UNIR)

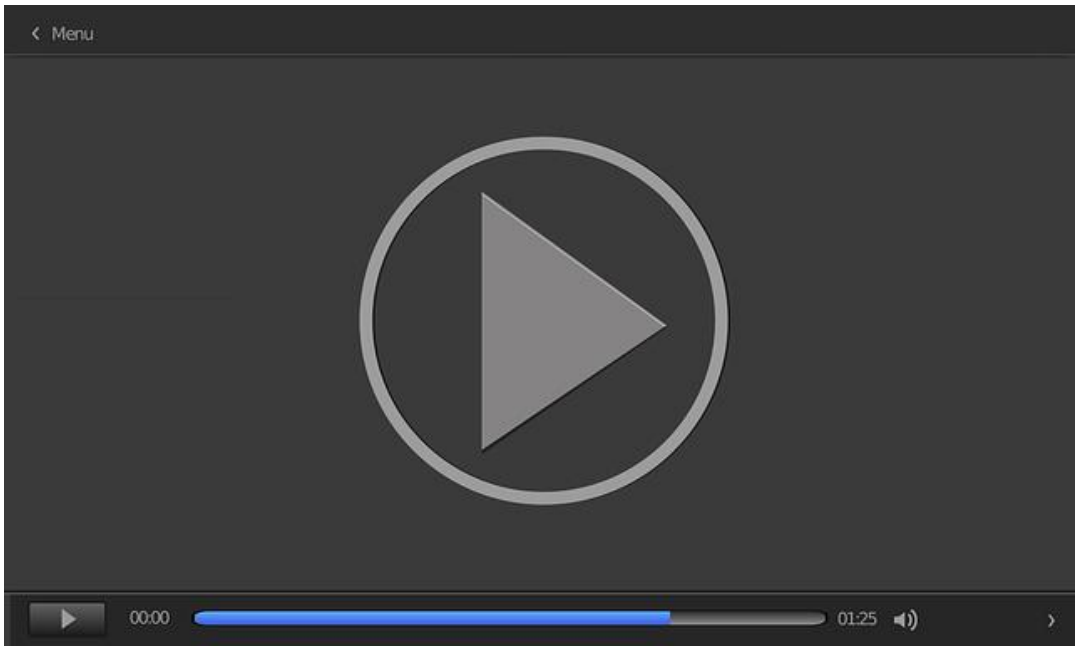
Ideas clave

- Limitar el **tamaño de las cabeceras y cuerpo** de los mensajes a un tamaño razonable.
- **Denegar peticiones** que tengan métodos **HTTP no soportados** (en principio, métodos distintos de POST).
- No habilitar conexiones persistentes.
- Utilizar un **servicio auxiliar** capaz de seguir respondiendo peticiones web incluso cuando los servidores están caídos, de modo que es posible entregar un mensaje informativo a los usuarios que comunique la indisponibilidad del servicio.
- Usar **estadísticas y registro** que muestran estadísticas en tiempo real y registran el estado, de modo que son útiles para detectar anomalías en la ratio de visitas web.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Se recomienda visualizar el vídeo *DDoS web attacks,* que repasa los principales ataques DoS web y medidas de protección.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=929b7d85-bcc3-437e-8cdf-ae5d00af8456

© Universidad Internacional de La Rioja (UNIR)

8.3. Mecanismos de protección

Para tener un nivel más de protección, de acuerdo con los principios de **seguridad de profundidad y diversidad** en la defensa, se puede complementar la defensa de la aplicación de varias formas. Además de utilizar las perimetrales a nivel de red y de las plataformas y sus sistemas operativos, existen:

- *Firewalls* de aplicaciones web (WAF).
- Herramientas de análisis dinámico en tiempo real (RASP).

Dos estudios comparativos sobre la eficacia de detección sobre herramientas WAF y RASP se puede consultar en Sureda (2017; 2022).

Web application firewall (WAF)

Los *firewalls* de aplicaciones web se interponen entre el cliente web y el servidor web **analizando los mensajes** de la capa de aplicación para detectar **violaciones en la programación de la política de seguridad** y **restringiendo el acceso** a ciertos puertos o servicios según lo configure el administrador.

Los WAF examinan peticiones y respuestas encapsuladas en los protocolos HTTP, SOAP, XML-RPC y capas de servicios web. Pueden efectuar **validaciones mediante expresiones regulares de tipo *whitelist, blacklist*** o una combinación de ambos. Se pueden configurar aplicando diferentes políticas de filtrado y pueden tener la característica de aprender de tráfico del pasado como **realimentación.** Tener políticas depuradas que minimicen el número de falsos positivos y negativos es una tarea complicada, necesita de una acertada programación de pruebas que permitan comprobar que no se ve afectada la funcionalidad ni el rendimiento.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Los **WAF permiten conexiones TLS,** para lo cual ha de instalarse en ellos una copia de la clave privada del servidor de aplicaciones que permita descifrar los datos por analizar. Posteriormente, estos se pueden enviar cifrados al servidor o en texto claro.

Cuando **detectan un patrón de ataque,** no redirigen las peticiones al servidor de aplicaciones o **terminan la conexión** directamente o a través de otro dispositivo (*router* o cortafuegos de red), al cual notifican la petición de bloqueo.

Atendiendo a su **diseño,** un WAF se clasifica por ser:

- Instalado en un ***hardware* específico** diseñado para una solución WAF concreta.
- Diseñado e implementado **directamente en el *hardware.***
- Instalado en un **servidor de propósito general.**

Los WAF permiten los siguientes **modos de instalación** (dependiendo del fabricante):

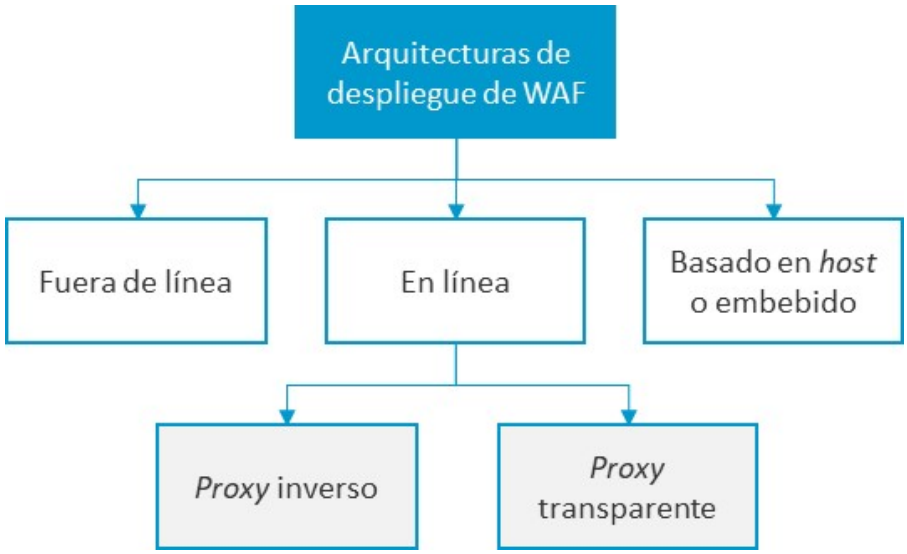


Figura 2. Modos despliegue de un WAF. Fuente: adaptado de Centro Criptológico Nacional, 2011.

© Universidad Internacional de La Rioja (UNIR)

- ***Proxy* inverso.** Suele ser el modo más habitual de conexión. El dispositivo examina todo el tráfico de red bloqueándolo si detecta algún ataque o reenviando la petición al servidor. Si detecta un ataque, lo registra. Se configura publicando la dirección de la interfaz externa del cortafuegos para redirigir el tráfico al servidor de aplicaciones. Para tener redundancia, se pueden poner dos dispositivos.

- **Modo pasivo (fuera de línea).** Se instala de forma que recibe un duplicado del tráfico vía *network tap,* de forma que pueda ser analizado y detectar ataques, pero sin posibilidad de bloquear.

- **Modo *bridge* (transparente).** Se instala de modo transparente, de forma que se comporta como un *switch* de nivel dos sin dirección IP. Esta forma de conexión podría permitir instalar un solo dispositivo sin cortar la comunicación con el servidor en caso de fallo, si se instala una tarjeta de red dual con *bypass*, aunque se pierde toda posibilidad de filtrado.

- **Embebido.** Se instala en el propio servidor como un *plugin*. Esta modalidad ha de ser bien estudiada, ya que puede tener incidencia en el rendimiento del servidor de aplicaciones.

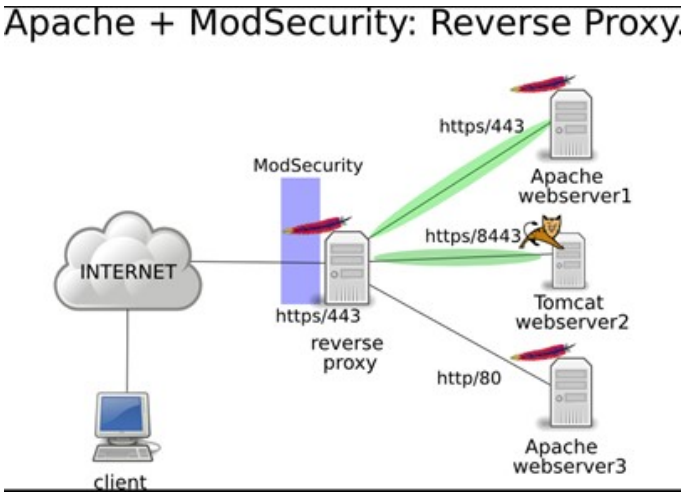


Figura 3. *Proxy* inverso con WAF ModSecurity basado en HOST *(plugin).*Fuente: Centro Criptológico Nacional, 2011.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Se puede consultar el proyecto Web Application Firewall Evaluation Criteria de WASC para estudiar detalladamente los requisitos que debe cumplir un dispositivo de este tipo (Turner, 2016) a través de este enlace: http://projects.webappsec.org/w/page/13246985/WebApplicationFirewall

El trabajo de Dawadi (2023) evalúa un modelo con **arquitectura en capas:** primero DDoS, luego XSS/SQL; consigue 97.57 % precisión para DDoS, ~89.34 % para XSS/SQLi y analiza también el impacto en latencia/*performance.* El trabajo de Domingues (2021) compara **modelos ML** con **motores basados en reglas.** Muestra que los clasificadores pueden ser mucho más rápidos que motores tradicionales de reglas (hasta ~18× menos tiempo de procesamiento) y mantener una precisión aceptable.

A modo de resumen:

- Hay que analizar el mercado y ver qué dispositivos se adaptan mejor a los requisitos de seguridad y disponibilidad de la instalación.
- Se debería instalar en modo de alta disponibilidad ante fallos mediante dos dispositivos.
- Se debe hacer un estudio del impacto en el rendimiento.
- Se deben probar las configuraciones de filtrado de forma que no impacten en la funcionalidad de las aplicaciones y que sean, a la vez, eficaces en cuanto a la detección de patrones de ataque. Son deseables siempre filtrados de lista blanca *(whitelist)* los que permiten solo tráfico conocido como aceptable, aunque implican un mayor estudio de complicaciones de la aplicación en cuanto a mantenimiento si esta sufre muchos cambios.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Hay que estudiar la forma y el nivel de detalle de registro de detecciones y bloqueos de forma que sean conformes con los requisitos de formato exigidos.
- Hay que gestionar y administrar los dispositivos con protocolos de conexión seguros.

WAF ModSecurity

Como ejemplo de WAF *open source*, se analizan las posibilidades de ModSecurity (Centro Criptológico Nacional, 2011), que trabaja en cinco fases de procesamiento que incluyen: encabezados del pedido *(request headers),* cuerpo del pedido *(request body),* encabezados de respuesta *(response headers),* cuerpo de respuesta *(response body)* y almacenamiento en bitácora *(logging):*

- Filtrado de peticiones: las peticiones HTTP entrantes son analizadas por el módulo mod_security antes de pasarlas al servidor web Apache y comparadas con un conjunto de reglas predefinidas para realizar las acciones correspondientes. Para realizar este filtrado, se pueden utilizar expresiones regulares, lo que permite que el proceso sea flexible.
- Técnicas antievasión: las rutas y los parámetros son normalizados antes del análisis para evitar técnicas de evasión.
- Elimina múltiples barras (//).
- Elimina directorios referenciados por sí mismos (./).
- Se trata de igual manera la \ y la / en Windows.
- Decodificación de URL.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

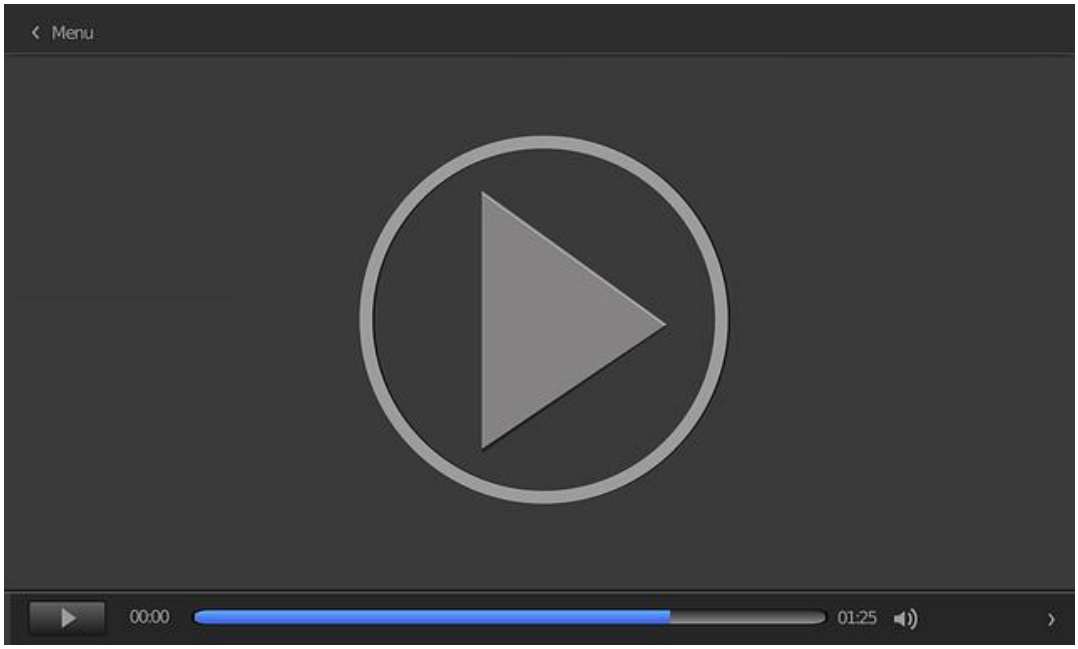
- Reemplazo de *bytes* nulos por espacios (%00).
- Comprensión del protocolo HTTP: al comprender el protocolo HTTP, ModSecurity puede realizar filtrados específicos y granulares.
- Análisis POST *payload*: intercepta y analiza el contenido transmitido a través del método POST.
- *Log* de auditoría: es posible dejar traza de auditoría para un posterior análisis forense.
- Filtrado HTTPS: al estar embebido como módulo, tiene acceso a los datos después de que estos hayan sido descifrados.
- Verificación de rango de *byte:* permite detectar y bloquear *shellcodes,* limitando el rango de los *bytes.*
- Soporte para *ranking* de anomalías y correlación básica de eventos (los contadores pueden ser automáticamente decrementados con el paso del tiempo).
- Soporte XML (*parseo*, validación).
- Bloqueo de IP.
- Detección de *malware* basado en web. Integración con antivirus.
- Protección de datos sensibles *(tracking).*
- Detección de acceso a troyanos.
- Detección de errores y ocultación para no mostrar más información de la necesaria, rutas, direcciones IP, etc.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Usa el conjunto de reglas de ModSecurity (Core Rule Set, CRS) alojado dentro del proyecto OWASP. Además, permite instalar y configurar el módulo Evasive para protección de ataques de denegación de servicio.

Se recomienda visualizar el vídeo *WAF* sobre los *firewalls* de aplicaciones web.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=2d596849-ad5c-4c57-93eb-ae5d00af8491

© Universidad Internacional de La Rioja (UNIR)

Real Analysis Self Protection (RASP)

RASP (Real Application Self-Protection) es una **tecnología de seguridad** que incorpora mecanismos dentro de la propia aplicación en ejecución para **detectar, prevenir** o **mitigar ataques.** Se basa en **instrumentación** en tiempo real, **monitoreo continuo** del flujo de datos, **eventos internos, llamadas** al sistema, etc., y puede actuar automáticamente para detener ataques que han eludido otras defensas externas (Seth et al., 2023). RASP funciona desde dentro del entorno de ejecución de la aplicación. A diferencia de otras soluciones, como los WAF (Web Application Firewalls), que analizan el tráfico desde el exterior, RASP monitorea el **comportamiento interno** de la aplicación en tiempo real, intercepta llamadas sensibles, valida entradas y toma acciones, como bloquear solicitudes o finalizar sesiones cuando detecta comportamientos maliciosos.

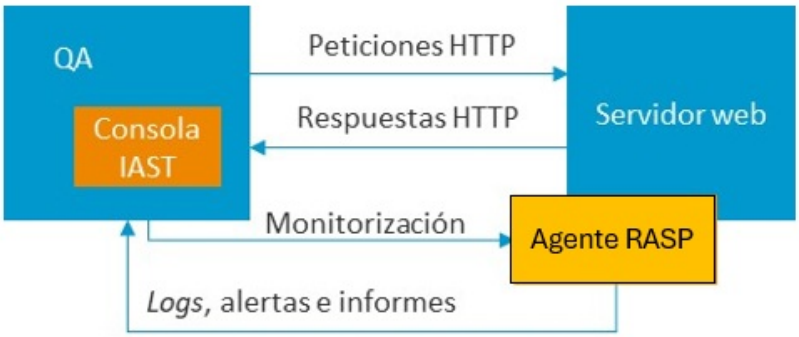


Figura 4. Arquitectura IAST. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

RASP se diferencia de WAF tradicionales (*firewalls* de aplicaciones) en que tiene **visibilidad interna** (contexto del código, estado de la aplicación) y puede intervenir desde dentro, no solo basándose en patrones de tráfico externos.

Modos de operación típicos:

- **Modo monitorización**: RASP observa, detecta, reporta, pero no bloquea. Se usa para pruebas, *tuning,* análisis de riesgos.
- **Modo protección**: bloquea o mitiga los ataques detectados (por ejemplo, abandonando la sesión, deshabilitando funcionalidad, cortando la conexión), además de reportar.

Elementos comunes de las herramientas RASP:

- **Instrumentación/*hooks*** en el código, librerías, *runtime* (por ejemplo, *frameworks, runtimes* de Java, .NET, Node.js, etc.).
- **Sensores que capturan eventos:** llamadas a funciones sensibles, consultas a bases de datos, validaciones de *inputs,* deserialización, acceso a archivos, etc.
- **Sistema de reglas**/heurísticas/modelos de comportamiento/análisis de flujos de datos.
- **Políticas de mitigación** automáticas cuando se detecta una anomalía o *exploit.*
- Registro, alertas, integración con **sistemas de monitoreo** SIEM/DevSecOps.

Forma de despliegue	Modo de integración	Ventajas	Desventajas
Agente/instrumentación en la aplicación	Se inyectan componentes (librerías, agentes) dentro del proceso de la aplicación, ya sea en el *runtime, framework,* contenedores o en la máquina virtual.	Alta visibilidad, acceso profundo al estado interno, posibilidad de detección detallada.	Puede añadir *overhead,* la compatibilidad con *frameworks*/versiones puede ser compleja, posible riesgo de afectar estabilidad si no está bien implementado.

Tabla 1. Despliegue RASP. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Pruebas/evaluación experimental

Evaluar una solución RASP implica medir tanto su **eficacia** (¿qué ataques detecta o previene?) como su **eficiencia/coste** (¿añade latencia? ¿cuánto *overhead* de CPU/memoria? ¿cuántos falsos positivos?).

Métricas usuales:

- Tasa de **detección** (verdaderos positivos).
- Tasa de **falsos positivos** y **falsos negativos.**
- **Tiempo de bloqueo** o reacción (latencia de respuesta tras detección).
- ***Overhead* en rendimiento:** CPU, memoria, tiempo de respuesta, *throughput.*
- **Cobertura de vectores de ataque** (XSS, SQLi, deserialización, *path traversal,* etc.).
- **Estabilidad del sistema:** compatibilidad, interferencia con funcionalidades legítimas.

Ventajas:

- **Protección en tiempo real** desde el interior de la aplicación, lo que permite una respuesta ante ataques que eluden defensas externas.
- **Visibilidad contextual** (estado del objeto, variables internas, flujo de datos), lo que mejora la precisión para distinguir entre tráfico legítimo y ataques.

© Universidad Internacional de La Rioja (UNIR)

- Puede ayudar a **mitigar vulnerabilidades** conocidas sin necesidad de parche inmediato *(virtual patching)* o mientras se corrige código.
- **Complementaotras técnicas** de seguridad: análisis estático, pruebas, *firewalls* externos, etc.
- En entornos móviles o distribuidos puede ser una capa adicional importante dada la **diversidad** de dispositivos, versiones y entornos de ejecución.

Desventajas/limitaciones

- Posible *overhead* de rendimiento: latencia, uso de CPU/memoria, impacto en *throughput.*
- Riesgo de **falsos positivos** si hay reglas muy estrictas y de **falsos negativos** si las reglas no cubren todos los casos o el agente/instrumentación no observa todos los caminos.
- **Complejidad de integración:** compatibilidad con versiones, *frameworks, runtimes,* lenguajes, componentes de terceros.
- Riesgo de que la **protección sea burlada** si el atacante controla partes del entorno de ejecución (ejemplo: *rooting, debugging, hooking* nativo).
- **Dependencia del buen diseño** del RASP, de mantenimiento continuo de las reglas/políticas y de pruebas frecuentes.

© Universidad Internacional de La Rioja (UNIR)

Recomendaciones prácticas para adopción

Para organizaciones que estén considerando usar RASP o reforzar su uso, estas son recomendaciones basadas en la literatura:

- **Iniciar en modo monitoreo/alerta:** primero, evalúa qué alertas se producen, cuantifica falsos positivos y ajusta políticas, antes de activar bloqueo automático.
- **Pruebas de compatibilidad** con todos los entornos de ejecución, versiones de *frameworks,* bibliotecas externas, módulos nativos, etc., para evitar que la protección interfiera con funciones legítimas.
- **Evaluación de rendimiento:** prueba RASP bajo carga con tráfico real, mide latencia, CPU, memoria. Debe asegurar que la degradación sea aceptable.
- **Cobertura de vectores de ataque:** asegúrate de que el RASP elegido soporte los tipos de ataque que son más probables o dañinos para tu aplicación (inyecciones, deserialización, *path transversal,* ataques lógicos, *business logic abuse,* etc.).
- **Mantenimiento y actualización de reglas/políticas:** dado que los ataques evolucionan, es importante tener un proceso para actualizar las reglas, añadir sensores, mejorar heurísticas, etc.
- **Seguridad del agente/instrumentación:** proteger el código del RASP, evitar que pueda ser deshabilitado por atacantes; asegurar la integridad del agente y uso de verificación de integridad si es posible.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Integración con *pipeline* de DevSecOps:** idealmente, RASP debe formar parte del proceso continuo de despliegue, pruebas de seguridad y monitoreo en producción; que los *feedbacks* se usen para mejorar.
- **Visibilidad y alertas:** asegurar que las alertas generadas por RASP lleguen al equipo adecuado, que haya *dashboards* o herramientas para analizar los eventos en contexto, distinguir gravedad, etc.

© Universidad Internacional de La Rioja (UNIR)

8.4. Operaciones de seguridad

Auditoría y monitorización continua

Una de las categorías de vulnerabilidades más frecuentes (OWASP, s. f.) y que aumentan el nivel de riesgo en las aplicaciones *online*, a la vez que retrasan la gestión de incidentes de seguridad, es el **registro y monitorización insuficientes.** Por un lado, las aplicaciones *online* deben diseñar correctamente el registro *(logging)* en fases de análisis y de diseño del ciclo de desarrollo del *software* (*secure software development life cycle,* SSDLC) para que los *logs* de todas las capas de la aplicación aporten **solamente la información necesaria** para detectar ataques o incidentes de seguridad, sin incluir aquella información superficial que no añade conocimiento, ni funcional ni de seguridad.

Por otra parte, hay que tener en cuenta que la actividad de monitorización puede ser tediosa y, en muchas ocasiones, se descuida y solo se lleva a cabo cuando se sospecha que algo no va bien. La **actividad de monitorización debe ser continua,** lo cual posibilitará **detectar de forma temprana** y anticiparse a posibles efectos adversos sobre la aplicación web.

La actividad de monitorización, lógicamente, se lleva a cabo en la **fase de producción** dentro del SSDLC y forma parte del procedimiento de **gestión de incidentes** de seguridad que debe diseñarse, probarse y ejecutarse debidamente cuando se detectan incidentes de seguridad.

© Universidad Internacional de La Rioja (UNIR)

Periódicamente, se debe **analizar la seguridad del código fuente** de la aplicación web y comprobar el *software* o librerías de terceros que usa para la detección de posibles vulnerabilidades y exposiciones comunes (*common vulnerabilities and exposures,* CVE). Las vulnerabilidades encontradas deben gestionarse adecuadamente y solucionarse, parchearse lo antes posible para evitar que puedan ser aprovechadas por algún atacante.

Existen herramientas, como OWASP Dependency Check, que permiten comprobar de forma automática la existencia de vulnerabilidades CVE. Puedes encontrar más información a través del siguiente enlace: https://owasp.org/www-project-dependency-check/

Así mismo, deben **auditarse de forma periódica** todas las **configuraciones de seguridad** de todas las capas de la aplicación, clientes, servidor de aplicaciones, aplicación web, servidor gestor de bases de datos y base de datos. El principio de mínimos privilegios en cuanto a la administración de permisos, roles y usuarios debe mantenerse y observarse de forma continua con base en la necesidad de conocer que tiene cada usuario de los recursos de la aplicación.

Gestión de incidentes de seguridad

Para poder gestionar con eficacia un evento de seguridad detectado hay que adoptar un ciclo que cuenta con las siguientes fases (CCN STIC817, 2012):

- **Preparación:** se reúnen las herramientas necesarias para el tratamiento del incidente (*antimalware,* comprobadores de integridad de ficheros o dispositivos, escáneres de vulnerabilidades, análisis de *logs,* sistemas de recuperación y *back-up,* análisis forense, etc.).

© Universidad Internacional de La Rioja (UNIR)

- **Identificación:** se detecta el incidente, se determina el alcance y se conforma una solución. Esta fase engloba a los responsables del negocio, operaciones y comunicación (contactos con soportes técnicos, equipo de respuesta ante emergencias informáticas, peritos forenses, policía o asesores legales si fueran necesarios, etc.).

- **Contención:** se impide que el incidente se extienda a otros recursos. Como consecuencia, se minimizará su impacto (separando equipos de la red afectada, deshabilitando cuentas comprometidas, cambiando contraseñas, etc.).

- **Erradicación:** se eliminan los elementos comprometidos, en caso de ser necesario (reinstalación de sistemas afectados o *back-up*), previos a la recuperación.

- **Recapitulación:** se documentan los detalles del incidente. Para ello, se archivarán los datos recogidos y se debatirán las lecciones aprendidas. Se informará a los empleados y se les enseñarán las recomendaciones dirigidas a prevenir situaciones de riesgo futuras.

Backup y recuperación

Para poder recuperar el estado correcto de una aplicación web que se ha visto sometida a un incidente de seguridad, es necesario disponer de un procedimiento de *back-up* y recuperación debidamente diseñado, probado y entrenado por los administradores de dicha aplicación.

El procedimiento de *back-up* y recuperación debe incluir:

- Copia de todos los archivos del proyecto de la aplicación web.

- Copia de todos los sistemas de ficheros del servidor de aplicaciones que contiene a la aplicación web.

© Universidad Internacional de La Rioja (UNIR)

- Copia de la base de datos de la aplicación lógica de las tablas, vistas, procedimientos almacenados, etc.

- Copia física de los discos donde residen todos los ficheros de la aplicación web, servidor de aplicaciones, servidor gestor de base de datos (archivos de control, *tablespaces* de sistema, archivos de red o *log,* etc.) y base de datos *(tablespaces)* de la aplicación web.

Esquema de protección de las aplicaciones web

Para lograr una **defensa efectiva** de las aplicaciones web de una organización, se deben tener en cuenta los principios de **profundidad y diversidad** de la defensa. Se debe implantar:

- **Protección específica** ante ataques DDoS utilizando dispositivos que detecten y separen el tráfico legal del ilegítimo. Lo ideal es que se combine la tecnología de equipo local del cliente con el servicio de operador para clientes del proveedor de servicios de Internet en un enfoque híbrido.

- Los beneficios del servicio híbrido anti-DDoS mejoran el control de la inversión, ya que no hay que contratar el servicio de operador con todos ellos, y minimizan los costes de operador, ya que el cliente puede mitigar los ataques hasta el ancho de banda disponible.

- Las desventajas del servicio se relacionan con que supone una inversión inicial para la instalación de equipos en el centro de proceso de datos del cliente y que requiere gestión local de los equipos y operación para la desconexión de los enlaces no protegidos cuando hay un ataque volumétrico que sature las líneas, así como la reconexión una vez se acabe el ataque.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

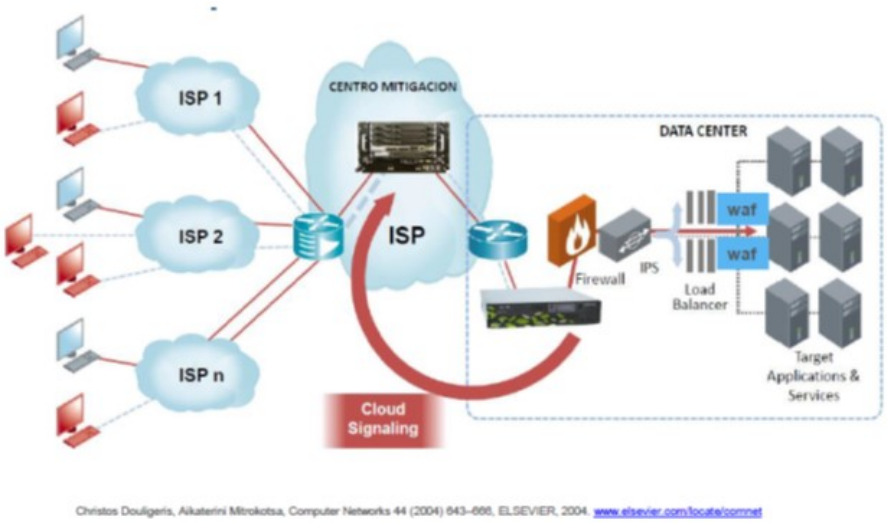
- **Cortafuegos de red** en modo de alta disponibilidad. Constituyen un primer nivel de defensa en el perímetro. No son suficientes, puesto que no filtran el contenido de los puertos en los que se despliegan las aplicaciones y servicios web.

- **Sistemas de gestión de eventos y correlación de *logs*** (*security information and event management*, SIEM) para integrar los *logs* de todos los dispositivos de defensa y *correlar* los eventos de seguridad que reportan. Deben tener monitorizadas todas las subredes donde se despliegan aplicaciones web mediante sistemas de detección de intrusiones y WAF.

- **Balanceadores de carga** para poder distribuir las peticiones al servidor de aplicaciones *clusterizado* que contiene las aplicaciones web.

- Sistema de **detección de intrusiones** para detectar ataques en general que tienen lugar en la red de la organización.

- **WAF en modo de alta disponibilidad.** Constituyen la protección principal *online* de las aplicaciones web. Se debe tener en cuenta la realización de las pruebas necesarias de eficacia de detección y rendimiento previo a su despliegue en fase de producción.



Christos Douligeris, Aikaterini Mitrokotsa, Computer Networks 44 (2004) 643–666, ELSEVIER, 2004. www.elsevier.com/locate/comnet

Figura 5. Profundidad y diversidad de la defensa. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

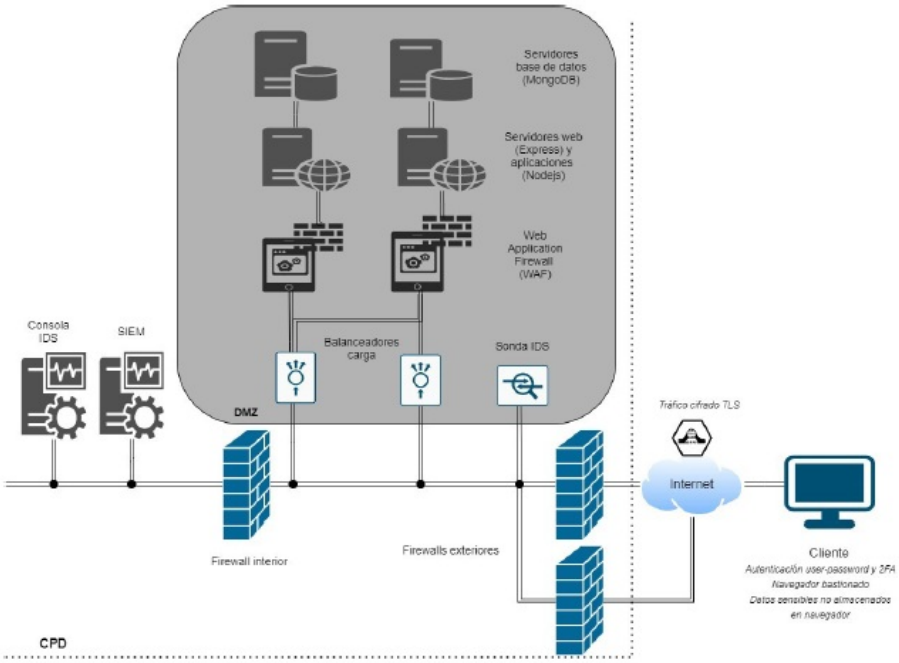


Figura 6. Profundidad y diversidad de la defensa. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

8.5. Referencias bibliográficas

Centro Criptológico Nacional (CCN). (2011). *CCN-STIC 661. Seguridad en Firewalls de Aplicación.* CCN-CERT. https://www.ccn-cert.cni.es/es/guias.html

Centro Criptológico Nacional (CCN). (2012). *CCN-STIC 817. Gestión de ciberincidentes.* CCN-CERT. https://www.ccn-cert.cni.es/es/guias.html

Dawadi, B. R., Adhikari, B. y Srivastava, D. K. (2023). Deep learning technique-enabled web application firewall for the detection of web attacks. *Sensors*, *23*(4), 2073. https://doi.org/10.3390/s23042073

Domingues M. J. y Ebecken, N. (2021). A new WAF architecture with machine learning
for resource- efficient use. *Computers & Security, 106*, 102290. https://doi.org/10.1016/j.cose.2021.102290

OWASP top ten web application security risks. (s. f.). https://owasp.org/www-project-top-ten/

Seth, A., Bhattacharya, S., Elder, S., Zahan, N. y Williams, l. (2023). Comparing effectiveness and efficiency of interactive application security testing (IAST) and runtime application self-protection (RASP) tools in a large Java-based system. *Comparing Effectiveness and Efficiency of IAST and RASP.* https://arxiv.org/abs/2312.17726

Sureda Riera, T. (2017). *Comparativa de la eficacia de herramientas WAF y RASP frente a ataques* [Tesis, Universidad Internacional de La Rioja]. Re-Unir. https://reunir.unir.net/handle/123456789/4742

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Sureda Riera, T., Bermejo Higuera, J. R., Bermejo-Higuera, J., Sicilia, J. A. y Martínez Herraiz, J. J. (2022). Systematic Approach for Web Protection Runtime Tools' Effectiveness Analysis. *Computer Modeling in Engineering and Sciences, 133*(3), 579–599. https://reunir.unir.net/handle/123456789/14433

Turner, T. (2016). *Web application firewall evaluation criteria.* The Web Application Security Consortium Project. http://projects.webappsec.org/w/page/13246985/WebApplicationFirewall

© Universidad Internacional de La Rioja (UNIR)

Firewall de aplicaciones web ModSecurity

Ristic, I. (2013). *ModSecurity handbook: the complete duide to the popular open source web application firewall.* Festy Duck. https://www.feistyduck.com/books/modsecurity-handbook/modsecurity-handbook-getting-started-may-2012.pdf

En este documento se explica cómo configurar el WAF ModSecurity.

© Universidad Internacional de La Rioja (UNIR)

Herramientas de protección en tiempo real (RASP)

Pathak, A. (29 de marzo de 2022). *7 Runtime Application Self-Protection (RASP) Tools for Modern Applications*. Geek Flare. https://geekflare.com/rasp-tools/

En este sitio web se relacionan varias herramientas de tipo RASP. Contrast Community Edition se activa desplegando un agente inteligente que instrumenta la aplicación con sensores inteligentes para analizar el código en tiempo real desde dentro de la aplicación. Es una versión de prueba gratuita. Para disponer de ella, hay que registrarse gratuitamente y se podrá monitorizar una aplicación web.

© Universidad Internacional de La Rioja (UNIR)

WAF contra RASP

Sureda Riera, T. (2017). *Comparativa de la eficacia de herramientas WAF y RASP frente a ataques* [Tesis, Universidad Internacional de La Rioja]. Re-Unir. https://reunir.unir.net/handle/123456789/4742

En este documento se comparan dos tecnologías de protección *online*, de caja negra WAF y de análisis RASP.

© Universidad Internacional de La Rioja (UNIR)

SANS Institute

Página web de SANS Internet Storm Center (https://isc.sans.edu/).

Cybersecurity Whitepapers. (s. f.). https://www.sans.org/white-papers

Membership & Community Resources. (s. f.). https://www.sans.org/security-resources

En SANS encontrarás diversa información relativa al estado actual de la seguridad, *papers*, cursos, certificaciones relacionadas con la asignatura, etc.

© Universidad Internacional de La Rioja (UNIR)

Normativa y noticias al día de ciberseguridad

Marco legal. (s. f.). https://www.ccn-cert.cni.es/gl/seguridad-al-dia/noticias-seguridad/normativa-legislacion.html

En este sitio web del CCN-CERT se pueden consultar las últimas novedades en cuanto a noticias y normativa legal relacionadas con la ciberseguridad.

© Universidad Internacional de La Rioja (UNIR)

Test

1. Un WAF detecta y bloquea ataques web usando:

   A. Expresiones regulares.

   B. Análisis semántico.

   C. No bloquea ataques.

   D. Ninguna de las anteriores.

2. ¿Qué tipo de instalación de un WAF solo detecta y no bloquea un ataque?

   A. *Bridge.*

   B. *Proxy* reverso.

   C. Pasivo.

   D. Ninguna de las anteriores.

3. ¿Qué tipo de instalación es la más recomendable para un WAF?

   A. *Bridge.*

   B. *Proxy* reverso.

   C. Pasivo.

   D. Ninguna de las anteriores.

4. El modo de instalación de un WAF que no necesita cambiar la configuración IP de la LAN de la organización es:

   A. *Bridge*.

   B. *Proxy* reverso.

   C. Pasivo.

   D. Ninguna de las anteriores.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Cuál es una forma de diseño de un WAF?

A. *Hardware* específico.

B. Embebido en el *hardware.*

C. Servidor de propósito general.

D. Todas las anteriores son ciertas.

6. Las defensas frente a ataques de denegación de servicio son:

A. Balanceadores de carga.

B. Clúster de servidores.

C. WAF.

D. Todas las anteriores.

7. Señalar la afirmación falsa:

A. Las herramientas de análisis de tipo IAST no suelen tener falsos positivos.

B. Las herramientas RASP no se pueden utilizar como *firewalls* de aplicaciones web de tipo *software*.

C. Las herramientas de tipo IAST, normalmente, tienen impacto en el rendimiento.

D. Las herramientas IAST pueden detectar únicamente, bloquear o sanear la petición.

8. Los WAF se pueden instalar como:

A. *Reverse proxy.*

B. Modo transparente.

C. Modo pasivo.

D. Todas las anteriores son ciertas.

© Universidad Internacional de La Rioja (UNIR)

9. Respecto a las variantes de instalación de los WAF:

A. La configuración en modo *proxy* inverso es la más deseable.

B. Se debe proporcionar redundancia en la configuración con dos WAF.

C. *Blacklist* consiste en mantener una base de datos de firmas de ataques, mientras que *whitelist* consiste en tener un modelo de tráfico aceptado entre la aplicación y el cliente.

D. Todas las demás son correctas.

10. Cuál de las siguientes afirmaciones describe con mayor precisión el funcionamiento de una herramienta RASP?

A. RASP analiza el código fuente de la aplicación antes de su ejecución para detectar vulnerabilidades.

B. RASP protege la aplicación inspeccionando únicamente el tráfico de red entrante desde el exterior.

C. RASP se integra en la aplicación en tiempo de ejecución para detectar y responder a amenazas desde el interior del sistema.

D. RASP se instala en el cortafuegos perimetral para bloquear ataques a nivel de red antes de que lleguen a la aplicación.

© Universidad Internacional de La Rioja (UNIR)